NESNEYE DAYALI PROGRAMLAMA

HAFTA 3

Java'da String ve Karakter İşlemleri

Bu bölümde, metinsel verileri (String) nasıl işleyeceğimizi, karakter bazlı kontrolleri nasıl yapacağımızı ve sayı sistemleri arasındaki dönüşümlerde bu metotlardan nasıl yararlanacağımızı göreceğiz.

Kullanacağımız Temel Metotlar:

- **length()**: String'in kaç karakterden oluştuğunu döndürür.
- **charAt(index)**: Belirli bir sıradaki karakteri verir (Unutmayın, sayma 0'dan başlar).
- **toUpperCase() / toLowerCase()**: Harfleri büyük veya küçüğe çevirir.
- **Character.isDigit(char)**: Karakterin bir rakam olup olmadığını kontrol eder.

Örnek 1

Kullanıcıdan alınan tek bir Hexadecimal basamağın (0-9 veya A-F), karşılık gelen 4 bitlik Binary değerini ekrana yazdıran bir Java programı tasarlayınız.

Kullanılacak Metotlar:

Programınızı oluştururken Java'nın şu temel yapı taşlarından faydalanacağız:

- **length()**: Kullanıcının sadece tek bir karakter girdiğinden emin olmak için.
- **toUpperCase()**: Küçük harf (a, b...) girişlerini büyük harfe (A, B...) sabitleyerek hata payını sıfıra indirmek için.
- **Integer.parseInt(s, 16)**: Metinsel (String) bir değeri 16'lık sayı tabanında tam sayıya dönüştürmek için.
- **Integer.toBinaryString()**: Sayıyı 0 ve 1'lerden oluşan ikilik sisteme dökmek için.

Örnek 1

Algoritma Kuralları: Programın doğru çalışması için şu 3 temel aşamayı geçmesi gerekir:

1. **Girdi Kontrolü:** Girilen metnin uzunluğu tam olarak **1** olmalıdır. Aksi halde kullanıcı uyarılmalıdır.
2. **Karakter Denetimi:** Girilen karakter sadece **0-9** arası bir rakam veya **A-F** arası bir harf olabilir.
3. **Dönüşüm Süreci:** Karakter önce 10'luk sisteme (Decimal) çekilmeli, ardından 2'lik sisteme (Binary) aktarılmalıdır.

Örnek Çıktı Beklentisi

- **Giriş:** B **Çıktı:** "Binary Değeri: 1011"
- **Giriş:** 5 **Çıktı:** "Binary Değeri: 0101"
- **Giriş:** G **Çıktı:** "HATA: Geçersiz bir hexadecimal karakter girdiniz!"

```java
public class HexToBinary {
    public static void main(String[] args) {
        // Kullanıcıdan veri almak için Scanner nesnesi oluşturulur
        Scanner input = new Scanner(System.in);

        System.out.print("Bir Hex basamağı (0-9 veya A-F) giriniz: ");
        // Kullanıcının girdiği metni String olarak okuyoruz
        String hexString = input.nextLine();

        // ADIM 1: length() metodu ile girişin uzunluğunu kontrol ediyoruz.
        // Hexadecimal basamaklar tek karakterli olmalıdır.
        if (hexString.length() != 1) {
            System.out.println("HATA: Lütfen sadece tek bir basamak giriniz.");
            return; // Programın çalışmasını burada sonlandırır
        }

        // ADIM 2: charAt(0) ile metnin ilk karakterini alıyoruz.
        // Character.toUpperCase() ile küçük harf girilse bile büyük harfe çeviriyoruz.
        char ch = Character.toUpperCase(hexString.charAt(0));

        // ADIM 3: Geçerlilik Denetimi
        // Karakterin A-F harfleri arasında mı yoksa 0-9 rakamları arasında mı olduğunu bakıyoruz.
        if ((ch >= 'A' && ch <= 'F') || (ch >= '0' && ch <= '9')) {

            // Integer.parseInt() metodu String beklediği için char'ı String'e çeviriyoruz.
            // 16 parametresi, Java'ya bu metnin 16'lık tabanda olduğunu söyler.
            int decimalValue = Integer.parseInt(String.valueOf(ch), 16);

            // Onluk tabandaki sayıyı (decimal), ikilik tabana (binary) çeviriyoruz.
            String binaryValue = Integer.toBinaryString(decimalValue);

            // Sonuçları ekrana yazdırıyoruz
            System.out.println("Girdi: " + ch);
            System.out.println("Binary Karşılığı: " + binaryValue);
        } else {
            // Eğer karakter A-F veya 0-9 dışında bir şeyse (Örn: 'G' veya '*') hata verir.
            System.out.println("HATA: '" + ch + "' geçersiz bir hexadecimal karakterdir.");
        }
    }
}
```

Örnek 2

Kullanıcıdan alınan 11 haneli bir T.C. Kimlik numarasının, aşağıda belirtilen matematiksel kurallara uygunluğunu denetleyen bir Java programı yazınız.

Programınızı yazarken aşağıdaki **String** ve **Character** sınıfı metotlarını kullanmanız gerekmektedir:

- **length()**: Girişin tam olarak 11 karakter olup olmadığını kontrol etmek için.
- **charAt(index)**: Belirli bir sıradaki rakama erişip matematiksel işleme dahil etmek için.
- **Character.isDigit(char)**: Kullanıcının girdiği karakterin bir rakam (0-9) olup olmadığını doğrulamak için.
- **Math veya % operatörü**: Mod alma işlemleri için.

Örnek 2

Algoritma Kuralları: Girilen numaranın geçerli sayılabilmesi için şu **3 temel kuralı** sağlaması gerekir:

1. **Sıfır Kuralı:** TC Kimlik numarasının ilk hanesi **0** olamaz.
2. **10. Hane Algoritması:** * 1, 3, 5, 7 ve 9. hanelerin toplamının **7 katından**,
   - 2, 4, 6 ve 8. hanelerin toplamı **çıkarılır**.
   - Elde edilen sonucun **10'a bölümünden kalan (mod 10)**, bize **10. haneyi** vermelidir.
3. **11. Hane Algoritması:** * İlk 10 hanenin (1'den 10'a kadar olan tüm rakamlar) toplamının **10'a bölümünden kalan**, bize **11. haneyi** vermelidir.

Örnek Çıktı Beklentisi:

- *Giriş:* 12345678901 *Çıktı:* "Geçersiz Algoritma"
- *Giriş:* 10000000146 *Çıktı:* "Geçerli TC Kimlik Numarası"

```java
public class TCKimlikKontrol {
    public static void main(String[] args) {
        // Kullanıcıdan giris almak için Scanner nesnesi baslatılır
        Scanner input = new Scanner(System.in);
        System.out.print("TC Kimlik Numarasını giriniz: ");

        // Giris metin (String) olarak alınır cünkü üzerinde karakter kontrolü yapacağız
        String tc = input.nextLine();

        // ADIM 1: length() metodu ile toplam hane sayısını kontrol ediyoruz
        if (tc.length() != 11) {
            System.out.println("Hata: TC Kimlik numarası tam 11 hane olmalıdır.");
            return; // Kural dısı giris olduğu için programı burada sonlandırıyoruz
        }

        // ADIM 2: charAt(0) metodu ile ilk karakterin '0' olup olmadığına bakıyoruz
        if (tc.charAt(0) == '0') {
            System.out.println("Geçersiz: TC Kimlik numarası '0' ile başlayamaz.");
            return;
        }

        // Matematiksel hesaplamaları kolaylaştırmak için rakamları bir int dizisine aktarıyoruz
        int[] n = new int[11];
        for (int i = 0; i < 11; i++) {
            // Character.isDigit() ile her karakterin bir rakam olup olmadığını denetliyoruz
            if (!Character.isDigit(tc.charAt(i))) {
                System.out.println("Geçersiz: TC Kimlik numarası sadece rakamlardan oluşmalıdır.");
                return;
            }
            // ASCII Karşılığı Farkı: Karakter olan '5'ten '0' cıkarılırsa sayı olan 5 elde edilir
            n[i] = tc.charAt(i) - '0';
        }
```

```java
        // ADIM 3: Matematiksel Kalıp (Algoritma) Kontrolü

        // 1, 3, 5, 7 ve 9. hanelerin toplamı (İndeksler: 0, 2, 4, 6, 8)
        int tekler = n[0] + n[2] + n[4] + n[6] + n[8];

        // 2, 4, 6 ve 8. hanelerin toplamı (İndeksler: 1, 3, 5, 7)
        int ciftler = n[1] + n[3] + n[5] + n[7];

        // 10. hane kuralı: (Tekler * 7 - Çiftler) mod 10
        int hane10 = (tekler * 7 - ciftler) % 10;

        // Mod işlemi bazen negatif sonuç verebilir, bunu pozitife çekmek için:
        if (hane10 < 0) hane10 += 10;

        // 11. hane kuralı: İlk 10 hanenin toplamının mod 10'u
        int toplam10 = 0;
        for (int i = 0; i < 10; i++) {
            toplam10 += n[i];
        }
        int hane11 = toplam10 % 10;

        // FİNAL KONTROLÜ: Hesaplanan değerler, girilen numaranın son iki hanesiyle eşleşiyor mu?
        if (n[9] == hane10 && n[10] == hane11) {
            System.out.println("Sonuç: " + tc + " GEÇERLİ bir TC Kimlik numarasıdır.");
        } else {
            System.out.println("Sonuç: TC Kimlik numarası algoritma kurallarına uymuyor (GEÇERSİZ).");
        }
    }
```